

Django Module 3 Important

1. What is a Template in Django? List any 3 basic template syntax with example and purpose. Illustrate how do you render templates in view.

What is a Template in Django?

A Template in Django is a text file (usually HTML) that defines how data is presented to the user. Templates combine static HTML with dynamic data provided by views.

Purpose:

- Separates presentation logic from business logic.
 - Displays dynamic content to users.
 - Supports reusable and maintainable UI design.
-

Three Basic Template Syntax

1. Variable Syntax

Syntax:

```
{{ variable_name }}
```

Example:

```
<h1>{{ name }}</h1>
```

Purpose:

- Displays the value of a variable passed from the view.
-

2. Tag Syntax

Syntax:

```
{% tag %}
```

Example:

```
{% if age >= 18 %}  
Adult
```

{% endif %}

Purpose:

- Performs logic such as conditions, loops, and template inheritance.
-

3. Comment Syntax

Syntax:

{# comment #}

Example:

{# This is a comment #}

Purpose:

- Adds comments that are not displayed in the browser.
-

Rendering Templates in a View

Template (home.html)

```
<!DOCTYPE html>
<html>
<body>
  <h1>Welcome {{ name }}</h1>
</body>
</html>
```

View (views.py)

```
from django.shortcuts import render

def home(request):
    data = {
        'name': 'Arun'
    }
    return render(request, 'home.html', data)
```

URL Mapping (urls.py)

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path("", views.home),
]
```

Working:

1. User requests a URL.
2. URL is mapped to a view.
3. View sends data to the template using `render()`.
4. Template displays the data.
5. HTML response is returned to the browser.

2. List and explain any 6 template tags with syntax, example and purpose.

Common Django Template Tags

Tag	Syntax	Purpose
if	{% if condition %}	Conditional execution
elif	{% elif condition %}	Additional condition
else	{% else %}	Default condition
for	{% for item in items %}	Loop through data
empty	{% empty %}	Executes if loop data is empty
extends	{% extends 'base.html' %}	Template inheritance
block	{% block content %}	Defines replaceable blocks
include	{% include 'file.html' %}	Includes another template
csrf_token	{% csrf_token %}	CSRF protection
url	{% url 'home' %}	Generates URL dynamically
load	{% load static %}	Loads template libraries
static	{% static 'css/style.css' %}	Access static files
comment	{% comment %}	Multi-line comments

with	{% with total=100 %}	Creates temporary variables
cycle	{% cycle 'odd' 'even' %}	Alternates values
firstof	{% firstof var1 var2 %}	Displays first non-empty value
autoescape	{% autoescape off %}	Controls HTML escaping
verbatim	{% verbatim %}	Prevents template processing
spaceless	{% spaceless %}	Removes whitespace between HTML tags
now	{% now "Y-m-d" %}	Displays current date/time

1. if

```
{% if age >= 18 %}
  Adult
{% endif %}
```

Purpose: Executes code when condition is true.

2. elif

```
{% if marks >= 80 %}
  Distinction
{% elif marks >= 60 %}
  First Class
{% endif %}
```

Purpose: Checks additional conditions.

3. else

```
{% if user %}
  Welcome
{% else %}
  Login Required
{% endif %}
```

Purpose: Executes when all conditions fail.

4. for

```
{% for student in students %}  
    {{ student.name }}  
{% endfor %}
```

Purpose: Iterates through collections.

5. empty

```
{% for student in students %}  
    {{ student.name }}  
{% empty %}  
    No Students Found  
{% endfor %}
```

Purpose: Executes if loop data is empty.

6. extends

```
{% extends 'base.html' %}
```

Purpose: Inherits parent template.

7. block

```
{% block content %}  
    Home Page  
{% endblock %}
```

Purpose: Defines editable sections.

8. include

```
{% include 'header.html' %}
```

Purpose: Reuses template components.

9. csrf_token

```
<form method="POST">
  {% csrf_token %}
</form>
```

Purpose: Protects against CSRF attacks.

10. url

```
<a href="{% url 'home' %}">Home</a>
```

Purpose: Generates URLs dynamically.

11. load

```
{% load static %}
```

Purpose: Loads template tag libraries.

12. static

```
{% load static %}

```

Purpose: Accesses static files.

13. comment

```
{% comment %}
This will not be displayed
{% endcomment %}
```

Purpose: Multi-line comments.

14. with

```
{% with total=students.count %}  
    Total Students: {{ total }}  
{% endwith %}
```

Purpose: Creates temporary variables.

15. cycle

```
{% for item in items %}  
    <tr class="{% cycle 'odd' 'even' %}">  
{% endfor %}
```

Purpose: Alternates values in loops.

16. firstof

```
{% firstof name username "Guest" %}
```

Purpose: Displays first non-empty value.

17. autoescape

```
{% autoescape off %}  
    {{ html_content }}  
{% endautoescape %}
```

Purpose: Controls HTML escaping.

18. verbatim

```
{% verbatim %}  
{{ variable }}  
{% endverbatim %}
```

Purpose: Prevents template processing.

19. spaceless

```
{% spaceless %}  
<p>Hello</p>  
<p>World</p>  
{% endspaceless %}
```

Purpose: Removes whitespace between HTML tags.

20. now

```
{% now "Y-m-d" %}
```

Purpose: Displays current date and time.

For exams, the most important tags are:

- `if`
- `for`
- `extends`
- `block`
- `include`
- `csrf_token`
- `url`
- `load`
- `static`

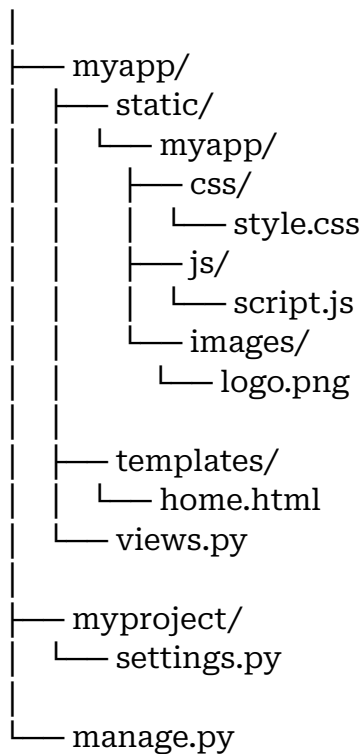
3. Outline a directory structure, configuration and loading static files into templates in Django.

Directory Structure, Configuration and Loading Static Files in Django

Static files are files such as CSS, JavaScript, and Images that are used to design and enhance web pages.

Directory Structure

myproject/



Configuration of Static Files

Step 1: Configure **settings.py**

```
STATIC_URL = 'static/'
```

Optional:

```
STATICFILES_DIRS = [
    BASE_DIR / "static",
]
```

- **STATIC_URL** specifies the URL prefix for static files.
- **STATICFILES_DIRS** specifies additional static file locations.

Loading Static Files in Templates

Step 1: Load Static Library

```
{% load static %}
```

Step 2: Use Static Files

CSS File

```
<link rel="stylesheet"
      href="{% static 'myapp/css/style.css' %}">
```

JavaScript File

```
<script src="{% static 'myapp/js/script.js' %}"></script>
```

Image File

```

```

Complete Template Example

```
{% load static %}
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <link rel="stylesheet"
```

```
        href="{% static 'myapp/css/style.css' %}">
```

```
</head>
```

```
<body>
```

```
    
```

```
    <script src="{% static 'myapp/js/script.js' %}">
```

```
</script>
```

```
</body>
```

```
</html>
```

Steps to Use Static Files

1. Create a **static** folder inside the app.
2. Store CSS, JavaScript, and image files inside it.
3. Configure **STATIC_URL** in **settings.py**.
4. Load the static library using:

```
{% load static %}
```

5. Access files using:

```
{% static 'path/to/file' %}
```

Purpose of Static Files

- CSS → Styling web pages
- JavaScript → Adding interactivity
- Images → Displaying graphics and logos

4. Show how to apply CSRF Token in Form with code snippets.

Applying CSRF Token in a Django Form

CSRF (Cross-Site Request Forgery) protection prevents unauthorized websites from submitting forms on behalf of authenticated users.

Step 1: Create a Form in Template

```
<form method="POST">

    {% csrf_token %}

    <label>Name:</label>
    <input type="text" name="name">

    <button type="submit">Submit</button>

</form>
```

Step 2: Create a View

```
from django.shortcuts import render
from django.http import HttpResponse

def student_form(request):

    if request.method == "POST":
        name = request.POST.get('name')
        return HttpResponse(f"Welcome {name}")

    return render(request, 'form.html')
```

Step 3: Configure URL

```
from django.urls import path
from . import views

urlpatterns = [
    path('form/', views.student_form),
]
```

Complete Template Example

```
<!DOCTYPE html>
<html>
<body>

<h2>Student Form</h2>

<form method="POST">

    {% csrf_token %}

    <input type="text" name="name" placeholder="Enter Name">

    <input type="submit" value="Submit">

</form>

</body>
</html>
```

Purpose of `{% csrf_token %}`

`{% csrf_token %}`

- Generates a unique security token.
- Inserts a hidden input field into the form.
- Django verifies the token when the form is submitted.
- Protects against CSRF attacks.
- Required for all POST forms.

Generated HTML (Browser View)

```
<input type="hidden"
      name="csrfmiddlewaretoken"
      value="random_generated_token">
```

If `{% csrf_token %}` is omitted in a POST form, Django raises:

403 Forbidden

CSRF verification failed. Request aborted.

5. Examine the working of Template Inheritance and blocks in Django with an example.

Template Inheritance and Blocks in Django

Template Inheritance allows one template to inherit the layout and structure of another template. It helps in reusing common HTML code such as headers, footers, and navigation bars.

`{% extends %}` and `{% block %}` tags are used for template inheritance.

Base Template (**base.html**)

```
<!DOCTYPE html>
<html>
<head>
  <title>Django Website</title>
</head>
<body>

  <h1>My Website</h1>

  {% block content %}
  {% endblock %}

</body>
</html>
```

Child Template (**home.html**)

```
{% extends 'base.html' %}

{% block content %}

<h2>Welcome to Home Page</h2>
<p>This content is from home.html</p>

{% endblock %}
```

View (**views.py**)

```
from django.shortcuts import render

def home(request):
    return render(request, 'home.html')
```

Working

1. User requests the home page.
2. Django loads `home.html`.
3. `home.html` extends `base.html` using:

```
{% extends 'base.html' %}
```

4. Django loads `base.html`.
5. The content inside:

```
{% block content %}  
{% endblock %}
```

of `base.html` is replaced by the content from `home.html`.

Output

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Django Website</title>  
</head>  
<body>  
  
  <h1>My Website</h1>  
  
  <h2>Welcome to Home Page</h2>  
  <p>This content is from home.html</p>  
  
</body>  
</html>
```

Purpose of `extends`

```
{% extends 'base.html' %}
```

- Inherits the parent template.
- Avoids duplication of common HTML code.
- Maintains consistent layout across pages.

Purpose of `block`

```
{% block content %}  
{% endblock %}
```

- Defines replaceable sections in the parent template.
- Child templates override these sections with their own content.

Advantages

- Code reusability.
- Easier maintenance.
- Consistent website design.
- Reduces duplication of HTML code.

6. List and explain any 6 template filters with syntax, example and purpose.

Template Filters in Django

Template filters are used to modify or format the value of variables before displaying them in templates. Filters are applied using the pipe (|) symbol.

Syntax

```
{{ variable|filter_name }}
```

or

```
{{ variable|filter_name:argument }}
```

Common Template Filters

Filter	Syntax	Example	Purpose
upper	<code>{{ value upper }}</code>	<code>{{ name upper }}</code>	Converts text to uppercase
lower	<code>{{ value lower }}</code>	<code>{{ name lower }}</code>	Converts text to lowercase
title	<code>{{ value title }}</code>	<code>{{ name title }}</code>	Converts text to title case
length	<code>{{ value length }}</code>	<code>{{ students length }}</code>	Returns length of string/list

truncatechars	{{ value truncatechars:10 }}	{{ msg truncatechars:10 }}	Truncates text after specified characters
default	{{ value default:"N/A" }}	{{ name default:"Guest" }}	Displays default value if empty
date	{{ value date:"d-m-Y" }}	{{ dob date:"d-m-Y" }}	Formats date
first	{{ value first }}	{{ names first }}	Returns first item
last	{{ value last }}	{{ names last }}	Returns last item
add	{{ value add:"10" }}	{{ marks add:"10" }}	Adds a value
join	{{ value join:", " }}	{{ names join:", " }}	Joins list items
slice	{{ value slice:":3" }}	{{ names slice:":3" }}	Returns a slice of data

1. upper

{{ name|upper }}

Input: arun

Output:

ARUN

Purpose: Converts text to uppercase.

2. lower

{{ name|lower }}

Input: ARUN

Output:

arun

Purpose: Converts text to lowercase.

3. title

{{ name|title }}

Input: arun kumar

Output:

Arun Kumar

Purpose: Converts text to title case.

4. length

{{ students|length }}

Output:

25

Purpose: Returns the number of items.

5. truncatechars

{{ message|truncatechars:10 }}

Output:

Welcome...

Purpose: Limits displayed text length.

6. default

```
{{ name|default:"Guest" }}
```

Output (if name is empty):

Guest

Purpose: Displays a default value.

7. date

```
{{ dob|date:"d-m-Y" }}
```

Output:

15-06-2003

Purpose: Formats date values.

8. first

```
{{ names|first }}
```

Output:

Arun

Purpose: Returns first element.

9. last

```
{{ names|last }}
```

Output:

Rahul

Purpose: Returns last element.

10. add

```
{{ marks|add:"10" }}
```

Output:

90

Purpose: Adds a value.

11. join

```
{{ names|join:", " }}
```

Output:

Arun, Rahul, Anjali

Purpose: Joins list elements into a string.

12. slice

```
{{ names|slice:":3" }}
```

Output:

['Arun', 'Rahul', 'Anjali']

Purpose: Returns a portion of a list or string.

7. Illustrate integration of bootstrap with Django templates.

Integration of Bootstrap with Django Templates

Bootstrap is a popular CSS framework used to create responsive and attractive web pages. It can be integrated into Django templates using a CDN link.

Step 1: Add Bootstrap CDN in Template

```
<!DOCTYPE html>
<html>
<head>
  <title>Bootstrap Example</title>

  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>

</body>
</html>
```

Step 2: Create a Django Template

home.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Django Bootstrap</title>

  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>

<div class="container mt-5">

  <h1 class="text-primary">
    Welcome {{ name }}
  </h1>

  <button class="btn btn-success">
    Submit
  </button>

</div>

</body>
</html>
```

Step 3: Create View

```
from django.shortcuts import render

def home(request):
    return render(request, 'home.html',
                  {'name': 'Arun'})
```

Step 4: Configure URL

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home),
]
```

Working

1. User requests the page.
2. View sends data to [home.html](#).
3. Bootstrap CSS is loaded through the CDN.
4. Bootstrap classes style the HTML elements.
5. The browser displays a responsive and styled page.

Common Bootstrap Classes

Class	Purpose
container	Creates responsive layout
btn	Creates button
btn-primary	Blue button
btn-success	Green button
text-primary	Blue text
text-danger	Red text
mt-5	Top margin
table	Styled table
form-control	Styled form input

Example Output

```
<h1 class="text-primary">
  Welcome Arun
</h1>
```

```
<button class="btn btn-success">
  Submit
</button>
```

The heading appears in blue and the button appears in green using Bootstrap styling.

8. List and explain built in form field types in Django.

Built-in Form Field Types in Django

Django provides various built-in form fields to accept and validate user input.

Field Type	Syntax	Purpose
CharField	forms.CharField()	Accepts text input
EmailField	forms.EmailField()	Accepts and validates email addresses
IntegerField	forms.IntegerField()	Accepts integer values
FloatField	forms.FloatField()	Accepts decimal numbers
BooleanField	forms.BooleanField()	Accepts True/False values
DateField	forms.DateField()	Accepts date values
DateTimeField	forms.DateTimeField()	Accepts date and time values
TimeField	forms.TimeField()	Accepts time values

ChoiceField	forms.ChoiceField()	Displays a dropdown list
MultipleChoiceField	forms.MultipleChoiceField()	Allows multiple selections
FileField	forms.FileField()	Accepts file uploads
ImageField	forms.ImageField()	Accepts image uploads
URLField	forms.URLField()	Accepts and validates URLs
PasswordInput	forms.CharField(widget=forms.PasswordInput)	Accepts password input
DecimalField	forms.DecimalField()	Accepts decimal values

Example

```
from django import forms
```

```
class StudentForm(forms.Form):
```

```
    name = forms.CharField(max_length=100)
```

```
    email = forms.EmailField()
```

```
    age = forms.IntegerField()
```

```
    percentage = forms.FloatField()
```

```
    dob = forms.DateField()
```

```
    website = forms.URLField()
```

```
    resume = forms.FileField()
```

```
    photo = forms.ImageField()
```

```
    department = forms.ChoiceField(
        choices=[
            ('MCA', 'MCA'),
            ('BCA', 'BCA')
        ]
    )
```

```
]
)
```

Explanation

CharField

```
name = forms.CharField(max_length=100)
```

- Accepts text input.

EmailField

```
email = forms.EmailField()
```

- Validates email addresses.

IntegerField

```
age = forms.IntegerField()
```

- Accepts integer values only.

FloatField

```
percentage = forms.FloatField()
```

- Accepts floating-point numbers.

BooleanField

```
agree = forms.BooleanField()
```

- Accepts True or False values.

DateField

```
dob = forms.DateField()
```

- Accepts date values.

ChoiceField

```
department = forms.ChoiceField(
    choices=[('MCA','MCA'), ('BCA','BCA')]
)
```


- Displays a dropdown list.

FileField

```
resume = forms.FileField()
```

- Uploads files.

ImageField

```
photo = forms.ImageField()
```

- Uploads images.

URLField

```
website = forms.URLField()
```

- Validates web URLs.

PasswordInput

```
password = forms.CharField(  
    widget=forms.PasswordInput  
)
```

- Masks password characters.